PROGRAMACIÓN ORIENTADA A OBJETOS

Etapa 2

SIMULADOR DE SISTEMA HOSPITALARIO

Delgado Jesús · Piñón Leslie · Sierra Daniel · Tavera Javier · Toledo Germán

Equipo 3 · Grupo 7

ETAPA 2

OBJETIVO Y ESTRUCTURA

Simular un sistema hospitalario (registro, consulta, tratamiento y alta) con herencia, clases abstractas, interfaces y polimorfismo.

Interfaz

AtencionMedica

Pacientes

Paciente,
PacienteGeneral,
PacienteEspecialidad

Personal

Medico, MedicoGeneral,
MedicoEspecialista,
Enfermero,
EnfermeroGeneral,
EnfermeroEspecialista,
JefeEnfermeria

Hospital

Sistema (lógica y main)

ETAPA 2

INTERFAZ ATENCIONMEDICA

«interface» AtencionMedica · Paciente darTratamiento()

▲ implements ▲

Medico **Enfermero**

- Medico.darTratamiento(): pasa al paciente de consulta (C) a tratamiento (T).
- Enfermero.darTratamiento(): pasa al paciente de T a curado (A).
- Cada subclase de enfermero lo sobrescribe con @Override.

ETAPA 2

PACIENTE

Clase abstracta. Comportamientos abstractos: solicitarConsulta() y getTipoAtencion(). Su único setter es setEstado().

PacienteGeneral

Atributo propio: motivoConsulta
@Override solicitarConsulta()
Se asigna a un médico general.

PacienteEspecialidad

Atributo propio: especialidad
@Override solicitarConsulta()
Se asigna a un médico de su misma especialidad.

ETAPA 2

MÉDICO

Clase abstracta que implementa AtencionMedica. Comportamiento abstracto: darConsulta(). darTratamiento() ya está implementado en la base.

MedicoGeneral

Atributo propio: estudiosOrdenados
@Override darConsulta()
Método propio: solicitarEstudiosGenerales()

MedicoEspecialista

Atributo propio: pruebaEspecializada
@Override darConsulta()
Método propio: solicitarPruebaEspecializada()

ETAPA 2

ENFERMERO

Clase abstracta que implementa AtencionMedica. Sus subclases sobrescriben darTratamiento().

EnfermeroGeneral

Atributo propio: tipoCuidado
@Override darTratamiento()
aplicarCuidadosGenerales()

EnfermeroEspecialista

Atributo propio: procedimientoAvanzado
@Override darTratamiento()
aplicarCuidadosIntensivos()

JefeEnfermeria

Atributo propio: turnoSupervision
@Override darTratamiento()
supervisarGuardia()

POLIMORFISMO

Estático

Sobrecarga: mismo nombre, distinto parámetro.
El compilador elige cuál usar.

```
asignarPaciente(Medico m)
asignarPaciente(Enfermero e)
```

Dinámico

Sobrescritura: la variable es de tipo Medico, pero Java ejecuta el método de la clase real del objeto.

```
Medico m = new MedicoGeneral(...);
m.darConsulta(p); // ejecuta MedicoGeneral
m = new MedicoEspecialista(...);
m.darConsulta(p); //
ejecuta MedicoEspecialista
```

Misma llamada, distinto resultado según el objeto real.

Una sola lista guarda todas las subclases.

SISTEMA

Restricciones

- Paciente general: médico general.
- Paciente de especialidad: médico de su especialidad.
- Enfermeros: pacientes en tratamiento de su especialidad.
- Límites: 10 por médico y 3 por enfermero.
- JefeEnfermeria no trata pacientes; supervisarGuardia() lista a los enfermeros de su sector.

Pseudocódigo

```
asignarPaciente(Medico m):
  para cada p en listaPacientes:
    si p.tipoAtencion = m.especialidad
    y m tiene menos de 10 pacientes:
      m.registroEnSistema(p)

asignarPaciente(Enfermero e):
  igual, con estado T y límite de 3
```

ETAPA 2

CONCLUSIÓN

Encapsulamiento

Atributos private y un setter justificado.

Herencia

Tres bases abstractas con código común.

Polimorfismo

Interfaz, sobrecarga y listas de tipo base.

ETAPA 2

GRACIAS POR SU ATENCIÓN